

Factored Guard-Effect Embedding for Capability Composition

PCCST503 Assignment 2 | Technical report

Second design | 4 October 2026

1. Problem definition

This report presents a Factored Guard-Effect Embedding (FGEE) for PCCST503 Assignment 2. The task is to numerically represent formally specified states, goals and capabilities while preserving the relationships needed for compatibility and ordered composition. The application model is $A=(S,C,S_I,G,R,K)$, and an atomic capability retains the type, inputs, outputs, preconditions, effects, constraints, resources, quality, reliability, availability and execution mechanism specified in the brief.

The design focuses on representation rather than planning. It accepts an explicitly supplied sequence and builds its composite contract. It neither searches for sequences nor redesigns the Assignment 1 planner. The executable implementation is a deterministic specification simulator; actual API, database and GUI adapters are outside this experiment.

Abstract: A structured vector combines finite-domain guard masks, write masks, assigned-value masks, typed port-domain masks, mechanism counts, resources and operational attributes. Composition regresses later guards through earlier assignments and merges effects by last write. Across three heterogeneous synthetic applications, 283/283 pair checks, 9408 atomic-state checks and 1344 composite-state checks agreed with a separate interpreter. Eight test methods passed, including 80 seeded generated sequences. Vectors contain 75-84 coordinates rather than a matrix over all complete states. Exactness holds for conjunctions of unary finite-domain predicates and constant assignments.

2. Design requirements

The representation must distinguish functionally different operations, relate operations to states, capture precondition-effect and input-output dependencies, support composites and relate them to goals. Functional similarity must be separate from directional composability. Alternative mechanisms should remain distinguishable even when their functional effect is the same. Cost, reliability, availability, risk, constraints and resource requirements must be represented or checked explicitly.

FGEE meets these requirements through role-separated numerical blocks and explicit operations. State variables have stable names and finite domains. Guards describe allowed initial values; effects describe written final values. Input and output domains occupy separate blocks. Mechanism and quality blocks preserve implementation distinctions but are excluded from functional similarity. Resource presence and availability are hard execution gates.

Formal fragment: all guards and goals are conjunctions of predicates over one variable at a time, supporting eq, ne, gt, ge, lt, le and in. Effects are deterministic constant assignments, with persistence for variables not written. Boolean, categorical and bounded numeric variables are supported. Cartesian products may be huge, but the core embedding never enumerates them. Relational predicates such as $\text{quantity} \leq \text{inventory}$ and arithmetic assignments require a richer extension.

3. Related embedding approaches

Word2Vec [1] motivates learning vectors whose geometry reflects relationships among words. Its context-based training does not certify a formal dependency such as Order=true before payment. Sentence-BERT [2] derives sentence vectors suited to cosine-based similarity. That is useful for retrieving semantically similar descriptions; the present task instead supplies formal guards, effects and contracts.

TransE [3] models relations as translations between entity embeddings. A fixed displacement is compact but does not itself distinguish guarded constant assignments from state-dependent updates. A factored one-hot representation is more interpretable for a known finite vocabulary, although its dimension grows with domain and schema sizes.

The composition calculus is inspired by predicate transformation and assignment substitution. Dijkstra [4] formalizes weakest preconditions and sequential composition. FGEE adapts that idea to a restricted, computable finite-domain mask operation and combines it with numerical interface and operational features. The custom contribution is the integrated capability vector and its exact regression, coverage and contract operations. It is not a learned linguistic embedding or a claim to invent weakest-precondition reasoning.

The evaluated baseline is a fixed threshold on our semantic similarity, not an implementation of Word2Vec, SBERT or TransE. A full finite-state transition vector is used only as an analytical storage reference: its size is quadratic in the number of complete states. No such matrix is allocated by FGEE.

4. Proposed representation

Let variables be $x_1, \dots, x_n$ with finite domains $D_1, \dots, D_n$. Set $D = \sum_k |D_k|$. A state is represented by concatenating one-hot vectors for each variable, using D coordinates rather than one coordinate for each complete state. A goal is represented by a mask G_k of permitted values for every variable; unconstrained variables permit their whole domain.

Each capability has an allowed-value set B_k for every variable, a write flag W_k and, when written, a constant A_k . Preconditions, capability constraints and global policies are intersected into B_k . If a constant assignment violates a global policy on that variable, the capability is made structurally impossible by an empty guard set. An empty guard anywhere means no valid source state.

The main functional vector is [guard masks, write flags, assigned-value masks]. The contract block holds allowed-value masks for inputs and outputs, plus required-input flags. Port slots are pairs of name and type label; their finite value vocabularies are built from the problem specification. A new subset of a known port domain can be encoded, enabling intersected external requirements.

The remaining blocks are mechanism counts, resource indicators and seven raw operational values. Atomic mechanisms are canonical JSON combinations of type and mechanism, represented by one-hot counts. Composites retain summed counts. The schema fixes every coordinate meaning and must accompany numerical arrays. `from_vector` reconstructs the capability blocks from numbers without needing the original capability specification.

5. Mathematical formulation

State encoding: $\phi_S(s) = \text{concat}_k \text{onehot}_{\{D_k\}}(s[x_k])$ in $\{0,1\}^D$. Goal encoding: $\phi_G(G) = \text{concat}_k 1_{\{v \in G_k\}}$, also in $\{0,1\}^D$. A state satisfies a goal iff every selected state value has mask value 1; equivalently their dot product equals n . State validity requires exactly one declared value per variable.

Guard compilation: $B_k = \{v \text{ in } D_k : \text{every predicate on } x_k \text{ in } P_i \text{ union } K_i \text{ union } K \text{ is true at } v\}$. Applicability is $s[x_k] \text{ in } B_k$ for every k . On applicability, $f_i(s)[x_k] = A_k$ if $W_k = 1$, otherwise $s[x_k]$. For unary global policies, assigned values are checked against the policy mask; unassigned variables preserve policy validity.

Capability encoding: $\phi_C(C) = [b, w, a, i, o, r, h, z, q]$. Here b and a each have D entries, w has n , i and o each have V port-value entries, r has B required-input entries, h has H mechanism counts, z has J resource entries and q has 7 operational entries. Therefore $d_C = 2D + n + 2V + B + H + J + 7$. The implementation block order is exactly this order.

Operational q order is time_ms, money, energy, risk, resource_cost, reliability, availability. Units are retained for accounting, not mixed into semantic similarity. A raw full-vector distance establishes numerical distinction but is dominated by units such as milliseconds and must not be interpreted as functional closeness.

Functional similarity: let U be the union of written variables. $E(i, j)$ is the fraction of variables in U written by both to the same value, or 1 when U is empty. For each variable, $J_k = |B_{i,k} \text{ intersect } B_{j,k}| / |B_{i,k} \text{ union } B_{j,k}|$, taking 1 for an empty union. Define $\sigma(i, j) = 0.7 E(i, j) + 0.3 \text{ mean}_k J_k$. This bounded symmetric score emphasizes effect resemblance; weights are declared choices, not fitted parameters. Unconstrained-variable agreement can inflate guard resemblance.

Directional coverage: for each k , $t_k = 1$ if an upstream write $A_{i,k}$ belongs to $B_{j,k}$ and 0 otherwise; if there is no write, $t_k = |B_{i,k} \text{ intersect } B_{j,k}| / |B_{i,k}|$. Set $\kappa(i, j) = \text{product}_k t_k$ for a nonempty upstream guard box, otherwise 0. This is the exact fraction of modeled upstream-applicable states accepted by the downstream guard, under uniform counting over the Cartesian product. It is not a probability of runtime success.

Typed output o matches required input p when names and type labels agree and $\text{domain}(o)$ is a subset of $\text{domain}(p)$. Let I_{ij} be true when every required downstream input is supplied by a matching upstream output. Guaranteed compatibility requires a nonempty upstream guard box, every assigned singleton or preserved upstream guard set contained in the downstream guard, and I_{ij} . Existential compatibility requires $\kappa > 0$ and I_{ij} . Availability and current resources are separate execution questions.

Direct goal relevance is the fraction of nontrivial goal-variable restrictions established by a write to an allowed goal value. It identifies direct contribution, not every indirect dependency. `goal_satisfied` applies a supplied capability or composite to an initial state and checks all goal predicates. Fixed-sequence leave-one-out evaluation identifies upstream steps needed for that given sequence; it performs no planning.

6. Capability composition model

`compose([C1, ..., Cm])` means execute C_1 first and C_m last. For two capabilities C then D , substitute earlier assignments into the later guard. For a variable written by C , require $A_{C,k} \text{ in } B_{D,k}$; a failed requirement makes the composition impossible. For an unwritten variable, intersect $B_{C,k}$ with $B_{D,k}$. Empty intersections also make the composition impossible. The composite retains the resulting initial-state guard box.

Composite effects use latest assignment: $A_{\text{comp},k} = A_{D,k}$ if D writes k , else $A_{C,k}$ if C writes k , else no write. Write masks take logical OR and assigned-value masks select the later nonzero assignment block. Thus composition is a nonlinear, role-aware operation on encoded masks, not vector addition or averaging. Numerical vectors can be decoded, composed and repacked without source specifications.

Correctness argument: if C writes x_k , its value before D is the known constant $A_{C,k}$, so the later guard reduces to a constant truth test. Otherwise the value remains the initial x_k and both guards must hold, exactly represented by intersection. Unary guards are independent across variables. Last-write effects equal sequential assignments. These facts prove equality of composite and sequential partial functions; induction extends it to any finite sequence. Semantic composition is associative but generally noncommutative.

Example: Compile requires stage=RAW and writes BUILT; Test requires BUILT and writes TESTED; Deploy requires TESTED, credentials=true and environment=PROD, then writes DEPLOYED. The composite requires stage=RAW, credentials=true and environment=PROD, plus size>0. It writes DEPLOYED. Simply unioning effects would retain conflicting categorical assignments; adding vectors would lose the initial guard and overwrite semantics.

Contracts are processed in order. Prior outputs bind later inputs when the typed domain inclusion test succeeds. Without such an output, an input remains external. Repeated external requirements use domain intersection, require matching types and combine required flags by OR. Same-name incompatible internal outputs cause rejection. Outputs are retained until another output with the same name overwrites them. Port domains describe guarantees; the simulator does not generate actual output values.

For serial operations, time, money, energy and resource_cost sum. Reliability is the product of per-step reliability under an independence assumption. Aggregate risk is $1 - \text{product}(1 - \text{risk}_i)$, also requiring independence. Availability is AND, resources are union and mechanism counts sum. Composition constructs a structural capability even when operational gates currently prevent execution. Associativity of the full vector is tested on the supplied valid contracts; arbitrary unresolved external/internal port aliasing can require a richer contract calculus.

7. Implementation

embedding.py implements FactoredEmbedding, Vector, encode, from_vector, compose, compatibility, apply, execute, goal_relevance, goal_satisfied and similarity. Guards use Python sets, and the public numerical encoding uses NumPy arrays. The core module has no complete-state enumeration. Evaluation enumeration exists only in experiments.py and generated-case tests.

encode(state,"state") and encode(goal,"goal") produce D-coordinate representations. encode(capability) compiles guards and effects into the full vector. encode(encoded_composite) returns the validated same-schema object. from_vector recovers a capability from its numerical blocks, checking shape, finite values, binary masks, write consistency, mechanism counts and operational bounds.

apply checks structural state validity and guard satisfaction, then applies constant assignments. execute additionally checks current resource presence, availability, external required inputs and provided optional input-domain membership. It is deterministic and conditional on declared effects; it does not sample success/failure. Port type labels govern matching; finite domain membership governs input-value checking.

The dataset generator writes explicit JSON states, goals, contracts, constraints, resource requirements, costs, reliability, availability and mechanisms. The experiment runner emits JSON summaries, complete pairwise CSV outcomes and NPZ numerical vectors. Tests cover contradictions, typed mismatch, negative numerical gates, schema errors, associative nesting, numerical-only roundtrip composition, and 80 generated three-step guard/write cases.

From the package directory run `python -m pip install -r requirements.txt`, `python make_dataset.py`, `python experiments.py` and `python -m unittest -v`. Report regeneration additionally needs requirements-report.txt and `python build_report.py`. The README contains a complete example, including reconstruction and composition from numeric vectors.

8. Experimental methodology

The synthetic dataset has three different structures. Commerce contains Boolean order/cart/authentication flags, categorical payment status and theme, a numeric quantity and notification state. It includes the exact required CreateOrder -> MakePayment versus CreateOrder -> CancelCart pattern. CreateOrder has no state precondition or capability constraint. MakePayment requires order=true, while CancelCart requires order=false.

Logistics has reservation, labelling, dispatch and warehouse-notification dependencies, a role restriction and quantity>0. Its goal requires both dispatched=true and notified=true. A three-step shipping chain reaches only half the goal; a four-step sequence includes the independent notification branch. Build uses categorical stage overwrites RAW -> BUILT -> TESTED -> DEPLOYED, later credential/environment requirements and a nontrivial global size>0 policy.

Experiment 1 tests every ordered capability pair. An independent conditional interpreter evaluates formal guards and assignments over all declared complete states. It provides applicability, coverage and guaranteed compatibility labels without using compiled masks. The baseline predicts compatibility when $\sigma \geq 0.8$, a fixed illustrative threshold. No training or threshold tuning occurs.

Experiment 2 compares every supplied composite against direct sequence interpretation on every state, tests left/right nesting, records raw vector-addition error, and evaluates goals and leave-one-out omissions. Experiment 3 compares API or computation implementations with DATABASE and GUI alternatives, using functional similarity and raw numerical distinction. Experiment 4 examines unrelated theme/audit/cache operations and indirect contributors.

Experiment 5 varies money preference $U_{\alpha} = \text{Rel-}\alpha * \text{money}$ for α in $\{0, 1, 10\}$, then changes Database presence and primary-implementation availability. This examines cost and reliability tradeoffs plus resource and availability gates. Runtime timings average 500 atomic encodings and 500 compositions per application; scaling times average 200 two-step compositions.

A separate scaling workload increases the count of Boolean variables from 10 to 160 with fixed small contracts and two operations. It records actual factored vector dimension, actual allocation size, timing and analytically calculated dense-transition storage. The huge Cartesian products are neither created nor allocated. Tests add 80 seeded random three-step unary-guard/assignment sequences over the build universe, including contradictions and policy violations.

The oracle covers all domain states, including unreachable ones. Exactness is therefore a correctness result over declared finite semantics, not prediction on unseen data. The data are synthetic and restricted to the supported fragment. Timing has no confidence intervals and depends on the Python/NumPy environment.

9. Results

All eight test methods passed. Guaranteed pair labels matched on 283/283 cases (100%). The similarity threshold baseline matched 184/283 (65.02%). No coverage discrepancies were detected. All 9408 atomic-state interpreter comparisons and 1344 composite-state comparisons agreed. Each problem had distinct full vectors for every atomic capability.

Commerce CreateOrder -> MakePayment had coverage 1 and compatible inputs, while CreateOrder -> CancelCart had coverage 0. The build Compile -> Test pair was guaranteed compatible; Compile -> ResetBuild was impossible. WrongTypedPayment, BadLabel and BadTest each had state coverage 1 but incompatible typed inputs. Thus guard compatibility cannot replace interface checking.

Logistics ReserveStock -> ReleaseStock was structurally compatible despite undoing reservation. This is an intentional counterexample: composability is different from usefulness for a chosen goal. API/computation

alternatives and DATABASE/GUI alternatives had functional similarity 1, while full-vector distances were about 20.0999 and 160.0126, primarily reflecting time and mechanism differences.

The commerce three-step and build three-step composites satisfied their goals. The logistics three-step composite was valid but did not satisfy the two-part goal; direct goal relevance was 0.5. Adding NotifyWarehouse produced a four-step composite with relevance 1 and full goal satisfaction. Removing any listed step from each supplied sequence prevented full goal satisfaction from its initial state. All tested nested full vectors were associative.

Each irrelevant operation had direct goal relevance 0. The first operation also had direct relevance 0 in these terminal goals, despite being necessary for the full chain. Adding the irrelevant step to the completed chain preserved goal success but increased cost. Direct relevance is therefore a local signal; sequence checks are needed to identify indirect contribution and redundancy.

At $\alpha=0$, the primary implementation was preferred by reliability. At $\alpha=1$, DATABASE was preferred. At $\alpha=10$, GUI was preferred. Removing Database excluded that implementation; making the primary implementation unavailable excluded it. All changes were explicit gates or declared preference changes, without modifying functional similarity.

FGEE vectors contained 81, 84 and 75 coordinates for commerce, logistics and build, respectively: 648, 672 and 600 float64 bytes. Their complete state universes contained 384, 384 and 192 states. A dense transition-only vector for those universes would require 1,179,648, 1,179,648 and 294,912 bytes. These analytical references exclude auxiliary matrix-model features.

Domain	Exact pairs	Baseline	d	Encode ms	Compose ms
commerce	121/121	83/121	81	0.0165	0.0340
logistics	81/81	47/81	84	0.0188	0.0426
build	81/81	54/81	75	0.0173	0.0325

10. Analysis

Capability identity: guard, write and assigned-value blocks distinguish modeled functions; mechanism and quality blocks distinguish implementations. Capability names are not identity coordinates. Logically equivalent guard syntax compiles to the same mask. Explicit writes are preserved even when a guard makes the assigned value already true, so the vector also records intentional assignment structure rather than being purely an extensional-function canonicalizer.

State relationship and constraints: compiled masks exactly represent applicability for unary finite predicates. Later credential and environment requirements regress to the build composite initial guard. Global size policy is checked on sources and assigned successors. Negative test cases exercise zero quantity, missing input, unavailable operations and missing resources.

Precondition-effect and input-output compatibility: exact coverage and universal subset tests separate directional dependencies from functional resemblance. The typed mismatch cases expose the limits of state-only checking. Similarity is suitable for alternative discovery within a schema, while compatibility and operational gates certify the appropriate modeled relationship.

Composition: guard regression and last-write effects preserve ordered behavior for every tested state. Numerical-only decoding tests demonstrate that composite construction does not require the original formal

specs after encoding, although the fixed schema is essential. Full-vector addition fails because guard masks must intersect or be discharged and overwritten effect values must be selected.

Goal relevance: direct effects identify terminal contributions, while composite and omission checks reveal indirect dependencies. The logistics branch shows that a valid chain can achieve a useful subgoal without achieving the whole goal. Resource availability and destructive release illustrate further distinctions between structural validity, current execution feasibility and goal suitability.

Operational properties: raw quality units are stored for transparent accounting. U_{α} is an explicit illustrative decision rule, not an empirical quality predictor. The change of winner with α shows why preference coefficients should be declared. Reliability and risk estimates depend on independence and do not measure external services.

Consistency: all checks pass across Boolean commerce state, branching logistics and categorical build overwrites. This is stronger structural variation than a pure renaming test but still within a narrow synthetic fragment. Efficiency: with fixed guard arity and bounded domains, compilation costs $O(\text{predicate-domain work})$; mask state operations cost $O(D)$. Guard/effect composition costs $O(mD)$ for m steps, plus contract processing and vector packing. Naive Python contract matching may be quadratic in port count; the experiment keeps contracts small.

Storage is $O(D+n+V+B+H+J)$, excluding retained source metadata. Complete-state count $N=\text{product}_k |D_k|$ does not appear in FGEE dimension. In the fixed-interface Boolean scaling workload, $d=5n+8$: 20 variables used 108 entries and 864 bytes, while a dense transition matrix would need about 8.8 TB. At 160 variables the factored vector used 808 entries and 6,464 bytes. These results rely on exact factorization of unary guards, not a learned compression of arbitrary state relations.

Boolean vars	Vector entries	Bytes	Compose ms
10	58	464	0.0224
20	108	864	0.0316
40	208	1664	0.0485
80	408	3264	0.0835
160	808	6464	0.1667

11. Limitations

The exactness proof applies only to conjunctions of unary predicates and deterministic constant assignments. Cross-variable constraints, disjunctions, conditional effects, arithmetic updates and nondeterministic transitions are not supported. A relation such as $\text{quantity} \leq \text{inventory}$ does not factor into independent per-variable masks in general. Adding those forms can require formulas, decision diagrams, solver checks or multiple guarded components.

Finite numeric domains are explicit values, not interval abstractions of an unbounded real space. Large domains increase D , even though the Cartesian product is avoided. Structured objects, subtype semantics and value-generation mechanisms require extensions. Schema vocabularies for mechanisms and port values are closed; new symbols require a rebuilt schema.

Interface matching uses exact name/type labels and finite domain inclusion. It does not infer aliases, automatically convert values or verify implementation output guarantees. Reused names, external-to-internal shadowing, optional inputs, arbitrary contract conflicts and temporal output scopes require careful modeling. The supplied contracts avoid ambiguous shadowing; roundtrip tests establish consistency for those supported contracts, not a universal contract theorem.

Resource presence is modeled, not consumable quantities, scheduling or parallel contention. Availability is a static snapshot, not $A(t)$. Serial costs add; real latency, cache effects and concurrency may differ. Reliability and risk formulas assume independence. `execute` performs deterministic conditional simulation, not retries, stochastic failure sampling or live API calls.

The dataset is synthetic and small for semantic evaluation. The scaling workload only increases independent Boolean variables with fixed interfaces. Eight regression methods and generated cases support implementation correctness but cannot prove arbitrary program behavior. There is no learned-model comparison, train/test split, real benchmark or observed service-quality evaluation.

Semantic similarity weights and the 0.8 baseline threshold are illustrative. Guard resemblance includes unconstrained variables, so scores may rise with irrelevant schema growth. Direct goal relevance ignores preserved satisfied goals and indirect support. Raw full-vector distances depend on operational units and should not rank functional alternatives.

12. Conclusion

The FGEE solution shows that a structured numerical representation can preserve capability identity features, guard/effect dependencies, typed interfaces, ordered composition and goal relationships without complete-state enumeration. Its composition operation is exact for the supported unary-guard and constant-assignment fragment. The heterogeneous experiments establish implementation consistency and expose cases where resemblance, composability and goal usefulness differ.

The primary research question is answered positively within this formal fragment. The advantage is compact, interpretable factorization with explicit operational accounting. The boundary is relational expressiveness: compactness comes from restricted structure. Future work should extend guarded formulas and richer port contracts, compare learned retrieval against exact verification, and evaluate realistic formal application datasets. Planner redesign remains outside this assignment.

References and reproducibility

[1] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. <https://arxiv.org/abs/1301.3781>

[2] Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 3982-3992. <https://aclanthology.org/D19-1410/>

[3] Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., and Yakhnenko, O. (2013). Translating Embeddings for Modeling Multi-relational Data. NeurIPS 26. <https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf>

[4] Dijkstra, E. W. (1975). Guarded commands, non-determinacy and formal derivation of programs. Communications of the ACM 18(8), 453-457. Author manuscript EWD472: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD04xx/EWD472.html>

[5] PCCST503 Assignment 2. Design of a Vector Embedding for Capability Composition. Supplied assignment brief, 13 pages.

```
{'python': '3.12.14', 'numpy': '2.3.5', 'platform': 'Linux-6.18.44-x86_64-with-glibc2.39'}
```

All numbers originate in `results/results.json`. Pair labels and scores are in `results/*_pairs.csv`, while NPZ files contain actual numerical embeddings. Running `experiments.py` reproduces the correctness checks and generates new environment-dependent timings.